

Aula 4 – SQL (Funções)

Álvaro G. Pagliari
alvaro.pagliari@unoesc.edu.br

- Revisão Aula Passada
 - Subconsultas
 - Exists
- Funções
- Exercícios





- Consultas aninhadas
 - Na Projeção
 - No From
 - No Predicado
 - IN
 - ANY
 - ALL
 - Exists





Funções





- Programação estruturada
 - Procedimentos
 - Funções
- Em BD
 - Geralmente Procedures contém rollback e Functions não
- PostgreSQL
 - Tudo a mesma coisa



```
CREATE [ OR REPLACE ] FUNCTION
name( [ [ argname] argtype] )
[ RETURNS tipo | [TABLE (cols)] ]
AS $$
[DECLARE var tipo;]
BEGIN
    operações
END;
$$ LANGUAGE plpgsql;
```

Exemplo – condicional

```
CREATE OR REPLACE FUNCTION numero_par (i int)
RETURNS boolean AS $$
DECLARE
    temp int;
BEGIN
    temp := i % 2;
    IF temp = 0 THEN RETURN true;
    ELSE RETURN false;
    END IF;
END;
$$ LANGUAGE plpgsql;

SELECT numero_par(3), numero_par(42);
```

Exemplo – laço FOR

```
CREATE OR REPLACE FUNCTION fatorial (i numeric)
RETURNS numeric AS $$
DECLARE
    temp numeric; resultado numeric;
BEGIN
    resultado := 1;
    FOR temp IN 1 .. i LOOP
        resultado := resultado * temp;
    END LOOP;
    RETURN resultado;
END;
$$ LANGUAGE plpgsql;

SELECT fatorial(42::numeric);
```


Exemplo – laço WHILE

```
CREATE OR REPLACE FUNCTION fatorial (i numeric)
RETURNS numeric AS $$
DECLARE temp numeric; resultado numeric;
BEGIN
    resultado := 1; temp := 1;
    WHILE temp <= i LOOP
        resultado := resultado * temp;
        temp := temp + 1;
    END LOOP;
    RETURN resultado;
END;
$$ LANGUAGE plpgsql;

SELECT fatorial(42::numeric);
```

Exemplo – SQL dinâmico

```
CREATE OR REPLACE FUNCTION recupera_funcionario(id int)
RETURNS funcionario AS $$
DECLARE
    registro RECORD;
BEGIN
    EXECUTE 'SELECT * FROM funcionario WHERE id = ' || id INTO
    registro;
    RETURN registro;
END;
$$ LANGUAGE plpgsql;

SELECT * FROM recupera_funcionario(1);
```

Exemplo – cursor

```
CREATE OR REPLACE FUNCTION total_salarios()  
RETURNS numeric AS $$  
DECLARE  
    registro RECORD; resultado numeric;  
BEGIN  
    resultado := 0.00;  
    FOR registro IN SELECT * FROM funcionario LOOP  
        resultado := resultado + registro.salario;  
    END LOOP;  
    RETURN resultado;  
END;  
$$ LANGUAGE plpgsql;  
  
SELECT total_salarios();
```



- E quando eu preciso retornar apenas um valor de uma consulta?

```
SELECT select_expressions INTO [STRICT] target FROM ...;  
INSERT ... RETURNING expressions INTO [STRICT] target;  
UPDATE ... RETURNING expressions INTO [STRICT] target;  
DELETE ... RETURNING expressions INTO [STRICT] target;
```





- E se precisar retornar uma tabela?

```
1 CREATE OR REPLACE FUNCTION retornaHospedes()  
2 RETURNS TABLE (  
3     cpf char  
4 ) AS $$  
5 BEGIN  
6     RETURN QUERY  
7     SELECT h.cpf FROM hospedes h WHERE h.datasai = current_date ;  
8  
9 END;  
10 $$ LANGUAGE plpgsql;
```





- E se precisar retornar uma tabela?

```
1 CREATE OR REPLACE FUNCTION retornaHospedes2()  
2 RETURNS TABLE (  
3     cpf_N char  
4 ) AS $$  
5 DECLARE  
6     tupla RECORD;  
7 BEGIN  
8  
9     FOR tupla IN SELECT h.cpf FROM hospedes h WHERE h.datasai = current_date LOOP  
10         cpf_N := tupla.cpf || ' novo';  
11         RETURN NEXT;  
12     END LOOP;  
13  
14 END;  
15 $$ LANGUAGE plpgsql;
```



- E quando eu preciso retornar apenas um valor de uma consulta?

```
BEGIN
    SELECT * INTO STRICT myrec FROM emp WHERE empname = myname;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE EXCEPTION 'employee % not found', myname;
    WHEN TOO_MANY_ROWS THEN
        RAISE EXCEPTION 'employee % not unique', myname;
END;
```



- E se eu quiser apresentar alguma informação para o usuário?

```
RAISE [ level ] 'format' [, expression [, ... ] ] [ USING option = expression [, ... ] ];  
RAISE [ level ] condition_name [ USING option = expression [, ... ] ];  
RAISE [ level ] SQLSTATE 'sqlstate' [ USING option = expression [, ... ] ];  
RAISE [ level ] USING option = expression [, ... ] ;  
RAISE ;
```

DEBUG, LOG, INFO, NOTICE, WARNING, and EXCEPTION



- E se eu quiser apresentar alguma informação para o usuário?

```
RAISE NOTICE 'Calling cs_create_job(%)', v_job_id;
```

```
RAISE EXCEPTION 'Nonexistent ID --> %', user_id  
    USING HINT = 'Please check your user ID';
```

```
RAISE 'Duplicate user ID: %', user_id USING ERRCODE = 'unique_violation';  
RAISE 'Duplicate user ID: %', user_id USING ERRCODE = '23505';
```



- E se eu quiser apresentar alguma informação para o usuário?

```
RAISE 'Duplicate user ID: %', user_id USING ERRCODE = 'unique_violation';  
RAISE 'Duplicate user ID: %', user_id USING ERRCODE = '23505';
```

Table A-1. PostgreSQL Error Codes

Error Code	Condition Name
Class 00 — Successful Completion	
00000	successful_completion
Class 01 — Warning	
01000	warning
0100C	dynamic_result_sets_returned
01008	implicit_zero_bit_padding
01003	null_value_eliminated_in_set_function
01007	privilege_not_granted
01006	privilege_not_revoked
01004	string_data_right_truncation



- Como controlo as EXCEPTION?

```
BEGIN
  SELECT * INTO STRICT myrec FROM emp WHERE empname = myname;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RAISE EXCEPTION 'employee % not found', myname;
  WHEN TOO_MANY_ROWS THEN
    RAISE EXCEPTION 'employee % not unique', myname;
END;
```



- Posso criar sub blocos?

```
RAISE NOTICE 'Quantidade aqui eh %', qtdade;  -- Imprime 30
qtdade := 50;
--
-- Cria um subbloco
--
DECLARE
    qtdade integer := 80;
BEGIN
    RAISE NOTICE ' Quantidade aqui eh %', qtdade;  -- Imprime 80
    RAISE NOTICE 'Quantidade externa eh  %', blocoexterno.qtdade;  -- Imprime 50
END;
RAISE NOTICE 'Quantidade aqui eh  %', qtdade;  -- Imprime 50
RETURN qtdade;
END;
$$ LANGUAGE plpgsql;
```

```
[ <<rotulo>> ]
[ DECLARE
    declarações ]
BEGIN
    comandos
END [ rotulo ];
```

- E os arrays?

```
1 CREATE OR REPLACE FUNCTION recebendoArray( VARIADIC numeros NUMERIC[] )
2 RETURNS integer AS $$
3 DECLARE
4     soma int; i int;
5 BEGIN
6     soma :=0;
7     FOREACH i IN ARRAY numeros LOOP
8         soma:= soma+i;
9     END LOOP;
10    RETURN soma;
11 END;
12 $$ LANGUAGE plpgsql;
13
14
15 SELECT recebendoArray(VARIADIC ARRAY [1,2,3]);
```

- E os arrays?

```
1 CREATE OR REPLACE FUNCTION retornaCPFs()  
2 RETURNS TEXT[] AS $$  
3 DECLARE  
4     resultado TEXT[];  
5     i RECORD;  
6 BEGIN  
7     resultado := ARRAY[]::TEXT[];  
8     FOR i IN SELECT cpf FROM clientes LOOP  
9         resultado := array_append(resultado, i.cpf::TEXT);  
10    END LOOP;  
11    RETURN resultado ;  
12 END;  
13 $$ LANGUAGE plpgsql;
```

- E os arrays?

```
1 CREATE OR REPLACE FUNCTION retornaCPFs2()  
2 RETURNS TEXT[] AS $$  
3 DECLARE  
4     resultado TEXT[];  
5     i RECORD;  
6 BEGIN  
7     resultado := ARRAY[]::TEXT[];  
8     FOR i IN SELECT cpf FROM clientes LOOP  
9         resultado := resultado || i.cpf::TEXT;  
10    END LOOP;  
11    RETURN resultado ;  
12 END;  
13 $$ LANGUAGE plpgsql;
```

Exercícios

- 1) Faça uma função que calcule o fatorial de um número n;
- 2) Uma prática utilizada durante o desenvolvimento de aplicações que interagem com bancos de dados é a de definir procedimentos ou funções responsáveis pela inclusão, alteração e exclusão de registros. Para as tabelas de Quartos e Clientes, crie funções que atendam a essas operações, respeitando as seguintes regras:

- a) no caso de inclusões, a função deverá retornar a chave primária do novo registro como resultado;

INSERT [] RETURNING col

- b) no caso de alterações, a chave primária cujo registro deverá ser modificado deverá ser passada como parâmetro (juntamente com os dados a serem modificados no registro). O retorno dessa função deverá ser nulo;
- c) no caso de exclusões, a chave primária cujo registro deverá ser removido deverá ser passada como parâmetro. O retorno dessa função deverá ser true se algum registro foi excluído, e false caso contrário.

GET DIAGNOSTICS linhasAfetadas = ROW_COUNT;



Exercícios

- 3) Crie uma função 'passaRegua' que deverá fechar a conta do cliente. A função deve receber como parâmetro o CPF do cliente e retornar o valor a ser pago pelo hospede. Esta função deve alterar a dataSai do hospede com a data atual. A função deve somar e retornar o valor gasto com a estadia (dias hospedado * preço do quarto).
- 4) Atualize a função anterior para também considerar o valor gasto em consumo de itens do cardápio pelo hospede durante a estadia.
- 5) Crie uma nova tabela chamada de tabela teste. A tabela deve possuir dois campos id (serial primary key) e texto (varchar (100)). Crie uma função que irá receber um inteiro como parâmetro, e esse inteiro corresponderá ao número de registros que você deve gerar para essa tabela.

Para gerar um string aleatória use "**MD5(random())::text**"

- 6) Faça uma função que recebe como parâmetro o nome de um item consumido por um hospede, a quantidade consumida e o CPF do hospede. Com essas informações você deve armazenar o consumo do hospede. Assim, você terá que buscar o ID do item consumido na tabela de cardápio, caso o item não esteja cadastrado você deve cadastrá-lo;



Exercícios

- 7) Crie uma função que retorne um array com a descrição de todos os itens do cardápio.
- 8) Crie uma função que realiza a reserva de um hospede. A função deve receber as datas onde deve ser feitas as reservas e o CPF do cliente. A função deve fazer as validações referentes a reserva para evitar que um quarto seja reservado para o mesmo dia, ou mesmo período bem como não deixar que um cliente tenha mais de uma reserva ativa por vez.
- 9) Faça uma função que irá calcular quanto lucro o hotel obteve em um determinado período. A função deve receber uma data inicial e uma data final, e deve calcular o valor de todas as diárias para este período bem como os itens consumidos, e retornar o valor total obtido.
- 10) Faça uma função que receba o CPF de um cliente como parâmetro e retorne uma string contendo o nome do cliente e seu telefone. Por exemplo: 'Nome, telefone'
- 11) Crie uma função que gere clientes. A função deverá receber como entrada o número de hóspedes que se deseja gerar.
- 12) Faça uma função que deve gerar hospedagens. A função deve receber um inteiro que define quantos registros de hospedagem vão ser criados. Para cada hospedagem você deve gerar um período aleatório. Uma hospedagem também pode gerar consumos, para cada hospedagem gere aleatoriamente alguns consumos, com esse número variando de 0 a 30.

